

План верификации ПО

Содержание

1 ВВЕДЕНИЕ	2
1.1 Цель	2
1.2 Область применения	2
1.3 Термины, определения и соглашения	2
2 СРЕДА ВЕРИФИКАЦИИ	3
2.1 Связь с другими процессами жизненного цикла ПО	3
2.2 Организация и ответственности	3
2.3 Независимость	4
2.4 Инструменты верификации	4
2.5 Методы верификации	4
2.5.1 Испытания	4
2.5.2 Рассмотрение	5
2.5.3 Анализ	5
2.5.4 Формальная инспекция	5
3 МЕРОПРИЯТИЯ ПРОЦЕССА ВЕРИФИКАЦИИ	5
3.1 Формальная инспекция	5
3.1.1 Входные данные	5
3.1.2 Критерии начала мероприятия	6
3.1.3 Процедура выполнения мероприятия	6
3.1.4 Критерии окончания мероприятия	7
3.1.5 Выходные данные	7
3.1.6 Исправление несоответствий	7
3.2 Разработка тестовых примеров	7
3.2.1 Входные данные	7
3.2.2 Критерии начала мероприятия	8
3.2.3 Процедура выполнения мероприятия	8
3.2.4 Критерии окончания мероприятия	9
3.2.5 Выходные данные	9
3.3 Испытания ПО (формальный прогон тестовых примеров)	9
3.3.1 Входные данные	9
3.3.2 Критерии начала мероприятия	9
3.3.3 Процедура выполнения мероприятия	9
3.3.4 Критерии окончания мероприятия	10
3.3.5 Выходные данные	10
3.4 Анализ покрытия кода тестами	10
3.4.1 Входные данные	10
3.4.2 Критерии начала мероприятия	10
3.4.3 Процедура выполнения мероприятия	10
3.4.4 Критерии окончания мероприятия	11
3.4.5 Выходные данные	11

4 КРИТЕРИИ ПЕРЕХОДА	11
---------------------	----

5 ИНСТРУКЦИИ ПОВТОРНОЙ ВЕРИФИКАЦИИ	11
------------------------------------	----

1 ВВЕДЕНИЕ

1.1 Цель

Данный документ определяет мероприятия, которые будут выполняться в рамках практикума ТПКС для достижения целей процесса верификации ПО системы отслеживания статусов кабинетов НИЯУ МИФИ.

1.2 Область применения

Требования и положения настоящего Плана распространяются на все работы по верификации и обязательны для всех участников проекта.

1.3 Термины, определения и соглашения

Термин	Определение, толкование
Сообщение о проблеме	Документ процесса УК ПО, содержащий описание несоответствия в данных ЖЦ ПО или в описании процессов ЖЦ ПО.
Запрос на изменение	Документ процесса УК ПО, созданный для внесения изменений в данные ЖЦ ПО с целью устранения несоответствия, описанного в СП.
Артефакт	Атомарное данное ЖЦ ПО. Артефактом может быть, например, требование или файл исходного кода
Конфигурация	Составное данное ЖЦ ПО, которое может включать в себя любые артефакты, агрегации и конфигурации
Формальная инспекция	Способ верификации документов, основанный на экспертной оценке их правильности, выполняемой одним или несколькими инспекторами, как правило, с использованием проверочных перечней.
Рассмотрение	Метод верификации, при котором данным жизненного цикла ПО дается экспертная оценка правильности.
Тестовый пример	Совокупность входных условий и значений входных параметров, ожидаемых результатов, критерия «Прошёл/Не прошёл» ("Pass/Fail").

Сокращение	Расшифровка
УК	Управление конфигурацией
ПО	Программное обеспечение
СП	Сообщение о проблеме
ЗИ	Запрос на изменение
ФИ	Формальная инспекция

2 СРЕДА ВЕРИФИКАЦИИ

2.1 Связь с другими процессами жизненного цикла ПО

Процесс верификации взаимодействует с другими процессами жизненного цикла ПО посредством процесса управления конфигурацией ПО, получая из него все входные данные и возвращая в него все выходные данные жизненного цикла ПО.

Входными данными процесса верификации являются следующие выходные данные процессов ЖЦ ПО:

- Планы проекта;
- Требования к ПО;
- Исходный код;
- Результаты испытаний.

Выходными данными процесса верификации являются:

- Сообщения о проблемах;
- Тестовые примеры и процедуры;
- Результаты прогона тестовых примеров (результаты испытаний);
- Записи о ФИ;
- Результаты анализов.

2.2 Организация и ответственности

В мероприятиях процесса верификации ПО принимают участие сотрудники проекта, имеющие различные роли в процессах ЖЦ ПО. Большинство мероприятий процесса верификации проводится сотрудниками группы верификации, сотрудники других процессов привлекаются к участию в формальных инспекциях. Роли, выделяемые в контексте выполняемых мероприятий процесса верификации ПО, приводятся в подразделах раздела 3.

2.3 Независимость

Независимость при выполнении мероприятий верификации обеспечивается на персональном уровне, то есть для каждого верифицируемого объекта из множества данных ЖЦ ПО сотрудник проекта, выполняющий мероприятие по верификации, не участвует в создании данного объекта.

Примеры:

- автор объекта формальной инспекции не может быть инспектором в данной инспекции;
- при проведении инспекции требований к ПО как целого инспектором не может быть ни один из авторов, участвовавших в разработке частей требований;
- автор требований и автор кода, реализующего требования, не могут быть авторами тестовых примеров для верификации этих требований;
- авторы требований, кода и тестовых примеров не могут быть авторами отчета об анализе покрытия кода тестами.

2.4 Инструменты верификации

Название инструмента	Предназначение инструмента
GitHub	Проведение формальной инспекции и рассмотрений, хранение документации к тестам
GitHub Actions	Система непрерывной интеграции для автоматического запуска тестов
Пакет testing (Go)	Пакет для модульного тестирования кода
Пакет testify (Go)	Пакет для интеграционного тестирования кода

2.5 Методы верификации

Для проведения мероприятий верификации будут использоваться методы, описанные ниже. Порядок применения этих методов детально описан в разделе 3.

2.5.1 Испытания

Испытания – метод верификации, при котором на вход ПО подаются определенные в тестовых примерах входные воздействия, а результат работы испытуемого ПО, полученный для данных входных воздействий, сравнивается со значениями, ожидаемыми на основании требований. Тестовые примеры разрабатываются на основе требований к ПО таким образом, чтобы обеспечить полное покрытие требований тестовыми примерами. В зависимости от совпадения результатов работы ПО с ожидаемыми значениями,

для каждого отдельного тестового примера определяется результат "Прошёл/Не прошёл" ("Pass/Fail").

Если существуют тестовые примеры с результатом "Не прошёл" ("Fail"), открываются сообщения о проблемах.

2.5.2 Рассмотрение

Рассмотрение – метод верификации, при котором данным жизненного цикла ПО дается экспертная оценка правильности. Все рассмотрения проводятся в форме формальных инспекций.

2.5.3 Анализ

Анализ – метод верификации, при котором проводится документальное подтверждение корректности анализируемых данных жизненного цикла ПО. Отличительным свойством анализа является воспроизводимость, т.е. результаты анализа, проведенного по одной и той же процедуре для одних и тех же данных жизненного цикла ПО, но разными людьми, будут идентичными.

Несоответствия, выявленные при проведении анализа, документируются в сообщениях о проблемах и таким образом передаются в процессы, результаты которых подвергаются анализу.

2.5.4 Формальная инспекция

Формальная инспекция – форма проведения рассмотрений и некоторых видов анализов. «Формальная» означает, что результат инспекции является основанием для изменения статуса документа в GitHub.

В ходе формальной инспекции одним или несколькими специалистами (инспекторами) осуществляется независимая проверка соответствия инспектируемых документов исходным документам. Для самоконтроля инспектора при проведении формальных инспекций, как правило, применяются заранее подготовленные и утвержденные проверочные перечни, различные для разных типов объектов инспекции. Ответы на вопросы проверочного перечня даются инспектором на основании экспертной оценки объектов формальной инспекции.

3 МЕРОПРИЯТИЯ ПРОЦЕССА ВЕРИФИКАЦИИ

3.1 Формальная инспекция

3.1.1 Входные данные

- План верификации ПО;
- Запрос на изменение;

- Объекты формальной инспекции;
- Входные данные процесса, результаты которого являются объектом формальной инспекции;
- Запись о формальной инспекции:
 - Титульный лист формальной инспекции;
 - Список замечаний (для повторной инспекции);
 - Проверочные перечни.

3.1.2 Критерии начала мероприятия

Одновременно выполнены следующие условия:

- Установлены базовые версии входных данных, на основе которых разработаны объекты формальной инспекции.
- По мнению автора, все изменения во все данные ЖЦ, требующие изменения по ЗИ, внесены.

3.1.3 Процедура выполнения мероприятия

Инициализация Автор помещает объекты инспекции в GitHub. Если формальная инспекция еще не проводилась, Автор создает запись о формальной инспекции из запроса на изменение, связывает ее с необходимыми проверочными перечнями, входными данными, на основании которых разрабатывались объекты инспекции, и справочными данными. GitHub автоматически устанавливает связь формальной инспекции с последними версиями всех единиц конфигурации, измененных по ЗИ, как с подлежащими инспекции версиями объектов инспекции.

Экспертиза и обсуждение Инспекторы проверяют существенные характеристики объектов инспекции, указанных на титульном листе записи о формальной инспекции. Обнаруженные несоответствия должны быть точно локализованы, сформулированы и записаны; при этом следует стремиться к таким формулировкам, которые идентифицируют проблемы, но не предлагают их решения. Каждое найденное несоответствие следует оформлять отдельно в списке замечаний записи о формальной инспекции в GitHub.

Регистрация результатов Ведущий завершает заполнение проверочных перечней, связанных с ФИ. При проведении повторной формальной инспекции ответы на вопросы проверочных перечней должны быть актуализированы, т.е. должны отражать состояние рассматриваемых версий объектов инспекции.

Если хотя бы одно из замечаний находится в состоянии «НАЗНАЧЕНО», Ведущий переводит запись о формальной инспекции в состояние «ПЕРЕРАБОТКА» ("REWORK"), в противном случае Ведущий переводит запись

о формальной инспекции в состояние «ЗАВЕРШЕНА» ("COMPLETED") и связанный с ней ЗИ в состояние «ЗАВЕРШЕН» ("COMPLETED").

3.1.4 Критерии окончания мероприятия

Выполнено одно из следующих условий:

- Формальная инспекция переведена в состояние «ЗАВЕРШЕНА» и связанный с ней ЗИ переведен в состояние «ЗАВЕРШЕН».
- Формальная инспекция переведена в состояние «ПЕРЕРАБОТКА».

3.1.5 Выходные данные

- Сообщения о проблемах;
- Запись о формальной инспекции (актуализированная):
 - Титульный лист формальной инспекции;
 - Список замечаний;
 - Заполненные проверочные перечни.

3.1.6 Исправление несоответствий

В процессе переработки объектов инспекции по замечаниям проведенной формальной инспекции для каждого зафиксированного в записи о формальной инспекции замечания в состоянии «НАЗНАЧЕНО»:

- Если Автор согласен с наличием зафиксированного несоответствия, он вносит необходимые изменения в объекты инспекции и переводит замечание в состояние «ИСПРАВЛЕНО».
- Если Автор считает, что исправление несоответствия нецелесообразно и несоответствие должно быть отклонено либо исправление должно быть отложено, то Автор снабжает замечание комментарием и переводит его в состояние «ОБСУЖДЕНИЕ».

3.2 Разработка тестовых примеров

3.2.1 Входные данные

- План верификации ПО;
- Требования к ПО;
- Запрос на изменение;
- Запись о ФИ тестовых примеров (если проводилась ФИ);
- Тестовые примеры (разработанные ранее).

3.2.2 Критерии начала мероприятия

Одновременно выполнены следующие условия:

- Базовые версии для артефактов тестируемых требований к ПО установлены.
- Формальная инспекция для изменений, выполненных по рассматриваемому ЗИ, не проводилась либо такая формальная инспекция завершена и находится в состоянии «ПЕРЕРАБОТКА».

3.2.3 Процедура выполнения мероприятия

Тест-инженер разрабатывает тестовые примеры для проверки соответствия реализации требованиям к ПО. Тестовые примеры должны покрывать как позитивные, так и негативные сценарии использования системы.

При разработке тестовых примеров следует создавать:

- Юнит тесты - для проверки отдельных модулей или компонентов системы в изоляции. Юнит тесты пишутся на языке Go с использованием пакета `testing` и библиотеки `testify/assert`. Каждый юнит тест представляет собой функцию с именем вида `TestXXX`, которая принимает параметр `t *testing.T`. Внутри функции выполняются проверки с помощью утверждений `assert`, например:

```
assert.Equal(t, expectedResult, actualResult, "Результаты должны совпадать")
```

Юнит тесты размещаются в файлах с суффиксом `_test.go` рядом с тестируемым кодом.

- Интеграционные тесты - для проверки взаимодействия между модулями системы. Интеграционные тесты также пишутся на Go с использованием пакетов `testing` и `testify`. Однако, в отличие от юнит тестов, они могут задействовать сразу несколько компонентов системы, в том числе внешние зависимости, такие как базы данных, очереди сообщений и т.д.

Для изоляции тестов от внешних зависимостей используются техники, такие как:

- Создание фейковых реализаций интерфейсов (заглушек)
- Поднятие контейнеров `Docker` со всеми необходимыми сервисами для теста
- Использование легковесных реализаций зависимостей (например, база данных в памяти)

Тестовые примеры должны быть автоматизированы там, где это возможно, чтобы обеспечить возможность регрессионного тестирования. Для запуска тестов используется либо непосредственно команда `go test`, либо CI/CD система (GitHub Actions).

Для выполнения тестов используется тестовый стенд - среда, максимально приближенная к промышленной, но изолированная от нее. Тестовый стенд должен быть сконфигурирован в соответствии с требованиями к аппаратному и программному обеспечению для корректного проведения испытаний. Конфигурация стенда описывается в виде кода (infrastructure as code) и хранится в репозитории вместе с исходным кодом проекта.

Если в ходе разработки тестовых примеров или их неформального прогона обнаружены несоответствия во входных данных процесса, то Тест-инженер должен открыть соответствующие СП.

По окончании разработки тестовых примеров Тест-инженер помещает их в GitHub в директорию с исходным кодом проекта.

3.2.4 Критерии окончания мероприятия

Тестовые примеры разработаны, найденные несоответствия во входных данных процесса описаны посредством СП. Тестовые примеры идентифицированы в GitHub.

3.2.5 Выходные данные

- Тестовые примеры;
- Сообщения о проблемах.

3.3 Испытания ПО (формальный прогон тестовых примеров)

3.3.1 Входные данные

- План верификации ПО;
- Исходный код;
- Тестовые примеры.

3.3.2 Критерии начала мероприятия

Базовые версии для исходного кода и тестовых примеров установлены.

3.3.3 Процедура выполнения мероприятия

Тест-инженер извлекает тестовые примеры из GitHub и осуществляет их формальный прогон с использованием соответствующей среды испытаний на базовой версии исходного кода.

Тест-инженер помещает результаты формального прогона в GitHub. Каждая версия результатов прогона должна трассироваться на версии тестовых примеров и версии исходного кода. Такую трассировку следует осуществлять созданием текстового файла с указанием идентификаторов теста, исходного кода и результатов прогона с последующим размещением этого файла в GitHub в одной агрегации с результатами прогона.

Если в ходе формального прогона тестовых примеров обнаружены несоответствия во входных данных процесса, то Тест-инженер должен открыть соответствующие СП.

3.3.4 Критерии окончания мероприятия

Тестовые примеры прогнаны с использованием соответствующей среды испытаний, найденные несоответствия во входных данных процесса описаны посредством СП. Результаты прогона тестовых примеров идентифицированы в GitHub.

3.3.5 Выходные данные

- Результаты испытаний ПО;
- Сообщения о проблемах.

3.4 Анализ покрытия кода тестами

3.4.1 Входные данные

- План верификации;
- Требования к ПО;
- Исходный код ПО;
- Тестовые примеры.

3.4.2 Критерии начала мероприятия

Все тестовые примеры разработаны, прогнаны с использованием соответствующей среды испытаний. Тестовые примеры и результаты их прогона идентифицированы в GitHub.

3.4.3 Процедура выполнения мероприятия

Инженер прогоняет тестовые примеры на инструментированной версии ПО, получая данные о покрытии кода тестами. На основе полученных данных инженер средствами GitHub формирует отчет о покрытии кода тестами.

Используя сформированный отчет, инженер проводит анализ покрытия. Каждый случай непокрытия элемента исходного кода следует отнести к одному из следующих классов и при необходимости открыть СП:

- Неполнота тестовых примеров. Следует открыть СП для принятия решения о модификации существующих или разработке новых тестовых примеров.
- Неполнота требований. Следует открыть СП для принятия решения либо об удалении кода, для которого не существует требований, либо о модификации существующих или разработке новых требований, позволяющих разработать основанные на требованиях тестовые примеры.
- Неиспользуемый код. Следует открыть СП для принятия решения об удалении кода.

При выполнении анализа инженер заполняет отчет об анализе покрытия кода тестами. В отчете должны быть идентифицированы версии требований к ПО, кода и тестов, использованных для получения данных о покрытии, перечислены случаи непокрытия и соответствующие СП.

3.4.4 Критерии окончания мероприятия

Отчет об анализе покрытия кода тестами создан и идентифицирован в GitHub, все найденные несоответствия зафиксированы в сообщениях о проблемах.

3.4.5 Выходные данные

- Отчет об анализе покрытия кода тестами;
- Сообщения о проблемах.

4 КРИТЕРИИ ПЕРЕХОДА

Детальные критерии для начала и завершения каждого из мероприятий, входящих в процесс верификации ПО, описаны в соответствующих подразделах "Критерии начала мероприятия" и "Критерии окончания мероприятия" различных секций главы "План верификации".

5 ИНСТРУКЦИИ ПОВТОРНОЙ ВЕРИФИКАЦИИ

Процесс управления конфигурацией ПО гарантирует, что для внесения любого изменения в данные жизненного цикла ПО должно быть создано сообщение о проблеме. При рассмотрении сообщений о проблемах, группа управления изменениями определяет самый ранний этап жизненного цикла ПО, где требуемое изменение начинает влиять на результаты выполняемых процессов жизненного цикла ПО и создает запросы на изменения для

этого этапа и всех последующих этапов жизненного цикла ПО, затрагиваемых изменениями. Список тестовых примеров, которые следует изменить и/или перепрогнать в связи с изменениями, определяется при рассмотрении сообщений о проблемах. Этот процесс гарантирует, что необходимые изменения будут внесены и верифицированы для каждого из этапов жизненного цикла, начиная с этапа, где изменение начинает влиять на результаты выполнения процесса.